APIs: POST and PUT

An API is an agreement between a client and a server. The client sends a request; the server validates it, does the work, and returns a response.

Learning goals

- Read an API route and its request/response shape.
- Explain the difference between GET, POST, PUT, and PATCH.
- Add a safe update route to the gallery API.

In this project

Express mounts the router under `/api`. For example, `router.get("/dishes")` becomes `GET /api/dishes`.

Route map

METHOD + PATH	PURPOSE	BRANCH STATUS
`GET /api/dishes`	List gallery entries as JSON.	Implemented
`GET /api/dishes/:id/image`	Return one image as image bytes.	Implemented
`POST /api/dishes`	Validate, inspect, and create a photo.	Implemented
`PUT /api/dishes/:id`	Replace editable details for one photo.	Class exercise — not implemented yet

Think of the request as a package

Method says what action to take. **Path** says which resource. **Headers** describe the format. **Body** carries data. The response includes a status code and, often, JSON.

POST vs PUT vs PATCH

METHOD	USE IT WHEN	TYPICAL RESULT
`POST`	Creating a new resource; server assigns its ID.	`201 Created`
`PUT`	Replacing the editable representation at a known ID.	`200 OK`
`PATCH`	Changing only some fields.	`200 OK`

Class convention: this exercise defines PUT as replacing the complete editable photo details (title and caption). It leaves the image and Inspector result unchanged. Use PATCH if the API should accept only one changed field.

READ THE CURRENT API · POST

What happens when a photo is created?

1 The browser sends JSON

The upload page resizes the selected image, then sends a data URL with the title and optional caption.

```
const response = await fetch("/api/dishes", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    title: "Sunday deep dish",
    caption: "Shared with the class",
    image: imageDataUrl
  })
});
```

2 The server validates and inspects

`DishUpload` checks the fields. `parseImageDataUrl` accepts JPEG, PNG, or WebP and enforces a 3 MB decoded-image limit. The Inspector checks that the photo shows Chicago deep dish.

3 The server saves only approved photos

Approved photos are inserted through `dbDishStore`. The gallery response contains metadata and an `imageUrl`; the image bytes are fetched separately.

4 The API responds

```
HTTP/1.1 201 Created
{
  "dish": {
    "id": 12,
    "title": "Sunday deep dish",
    "caption": "Shared with the class",
    "inspectorNote": "Inspector: looks like deep dish (92% sure)",
    "createdAt": "2026-11-10T12:00:00.000Z",
    "imageUrl": "/api/dishes/12/image"
  }
}
```

STATUS	MEANING IN THIS POST FLOW
201	Photo passed checks and was created.
400	Invalid fields, image format, or image size.
422	Inspector rejected the image as not a confident deep-dish match.
502	Inspector/model service failed; no photo is saved.

Keep secrets on the server

The browser never receives `GEMINI_API_KEY`. The API server reads it from Replit Secrets and calls Gemini. Never put API keys in frontend JavaScript or example requests.

CLASS EXERCISE · ADD PUT

Update a photo's details

The Activity 3 branch does not yet have a PUT route. Implement one that replaces a photo's title and caption while preserving the image and Inspector note.

1 Define the complete editable body

In `artifacts/api-server/src/routes/dishes.ts`, add Zod validation. In this PUT exercise, both fields are required.

```
const DishDetailsUpdate = z.object({
  title: z.string().trim().min(1).max(80),
  caption: z.string().trim().max(280)
});
```

2 Add an update operation to the store

In `DishStore`, define `update(id, details)` and return the updated row or `undefined`. Implement it in both stores: update the matching array entry in memory, and use Drizzle `db.update(...).set(...).where(...).returning()` for PostgreSQL.

3 Add the Express route

```
router.put("/dishes/:id", async (req, res): Promise<void> => {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id < 1) {
    res.status(400).json({ error: "Invalid photo ID." });
    return;
  }
  const parsed = DishDetailsUpdate.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Provide a title and caption." });
    return;
  }
  const updated = await store.update(id, parsed.data);
  if (!updated) {
    res.status(404).json({ error: "Photo not found." });
    return;
  }
  res.json({ dish: toGalleryItem(updated) });
});
```

Try it from the browser

```
await fetch("/api/dishes/12", {
  method: "PUT",
  headers: { "Content-Type": "application/json"
},
  body: JSON.stringify({
    title: "Updated title",
    caption: "Updated caption"
  })
});
```

Verify these cases

- Existing ID + valid body → `200`, updated dish.
- Bad ID or invalid body → `400`.
- Unknown ID → `404`.
- GET the gallery again and confirm changes persisted.

Instructor check: POST creates and assigns an ID; PUT targets a known ID and replaces the chosen editable fields. Never trust client input: validate it, check for a missing row, and return the right status.